2

Simple Values and Types

Now that you have had a short tour of Xcode in the previous chapter, let's look at the Swift programming language, which you will use to write your app.

First, you'll explore Xcode **playgrounds**, interactive environments where you can type in Swift code and have the results immediately displayed. Then, you'll study how Swift represents and stores various types of data. After that, you'll learn about **type inference** and **type safety**, which helps you to write more concise code and avoid common errors. Next, you'll learn how to perform common operations on data. Finally, you'll learn how to print messages to the Debug area to help you troubleshoot issues.

By the end of this chapter, you will be able to write simple programs that can store and process letters and numbers.

The following topics will be covered:

- Xcode Playgrounds
- Data types
- Constants and variables
- Type inference and type safety
- Operators
- The `print()` statement



For more information about the latest version of the Swift language, visit `https://docs.swift.org/swift-book/documentation/the-swift-programming-language/`.



To see the latest updates to the Swift language announced by Apple during WWDC 2026, watch this video: `https://developer.apple.com/videos/play/wwdc2026/262/`.

Technical requirements

To do the exercises in this chapter, you will need the following:

- An Apple Mac computer running macOS 26 Tahoe ~~26~~ or macOS 27 Golden Gate
- Xcode 27 installed (refer to *Chapter 1*, *Hello, Xcode*, for instructions on how to install Xcode)

The Xcode playground for this chapter is in the `Chapter02.zip` file of the code bundle for this book. To access the repository link, follow the steps in the *Download the example code files* section in the *Preface*.

Check out the following video to see the code in action:

To be added later

In the next section, you'll create a new Xcode playground, where you can type in the code presented in this chapter.

Xcode Playgrounds

Xcode Playgrounds are interactive coding environments. You type code in the left-hand area, and the results are immediately displayed in the right-hand area. It's a great way to experiment with code and explore the iOS SDK.



SDK is an acronym for software development kit. To learn more about the iOS SDK, visit `https://developer.apple.com/ios/`.

Let's start by creating a new playground and examining its user interface. Follow these steps:

1. To create a playground, launch Xcode and choose **New Project** | **Playground...**:

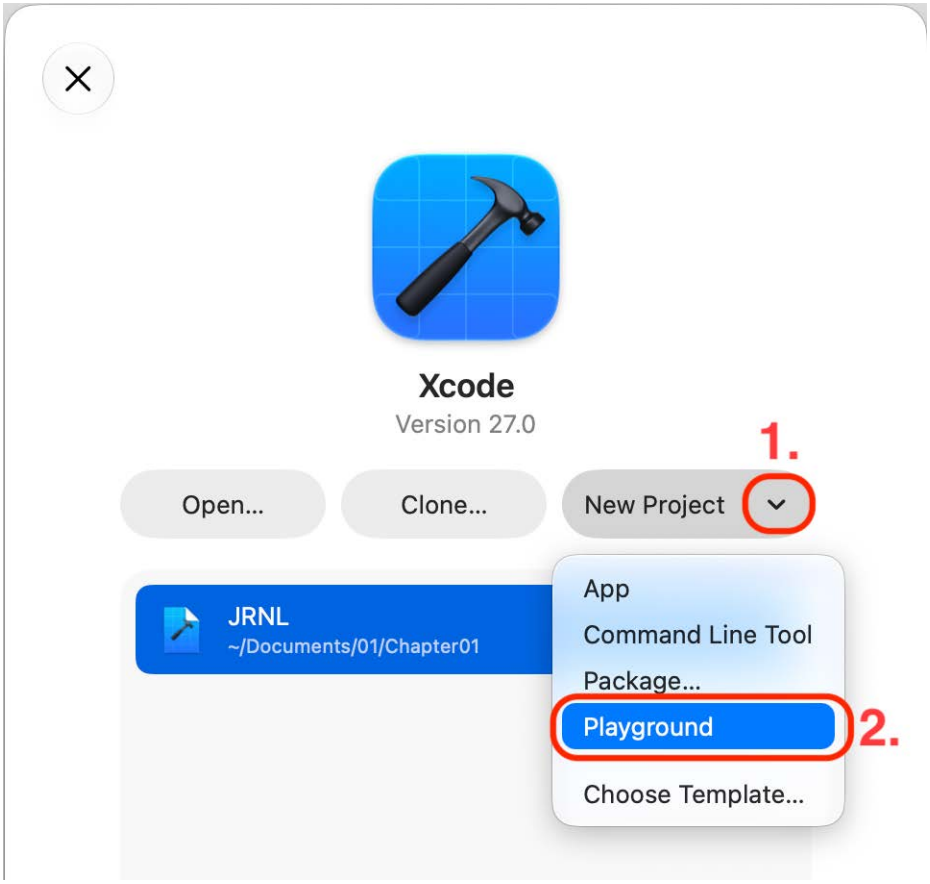


Figure 2.1: Xcode window showing New Project | Playground... selected



You can also choose **New | Project... | Playground** from the Xcode **File** menu.

2. The playground screen appears. Choose **File | Save File...**:

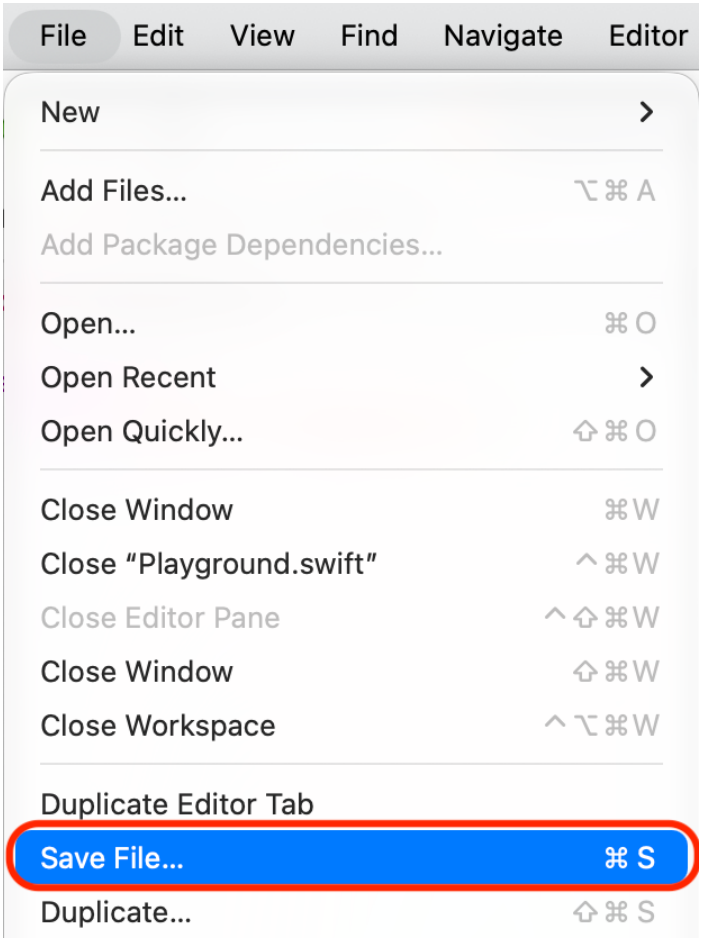


Figure 2.2: Xcode File menu with Save File... selected

3. Name your playground `SimpleValues` and choose a save location on your Mac. Click **Save** when done:

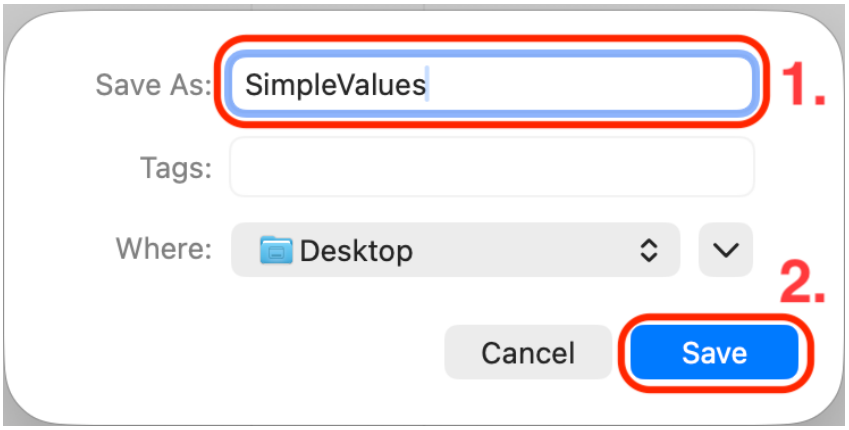


Figure 2.3: Save dialog box

4. You'll see the playground on your screen:

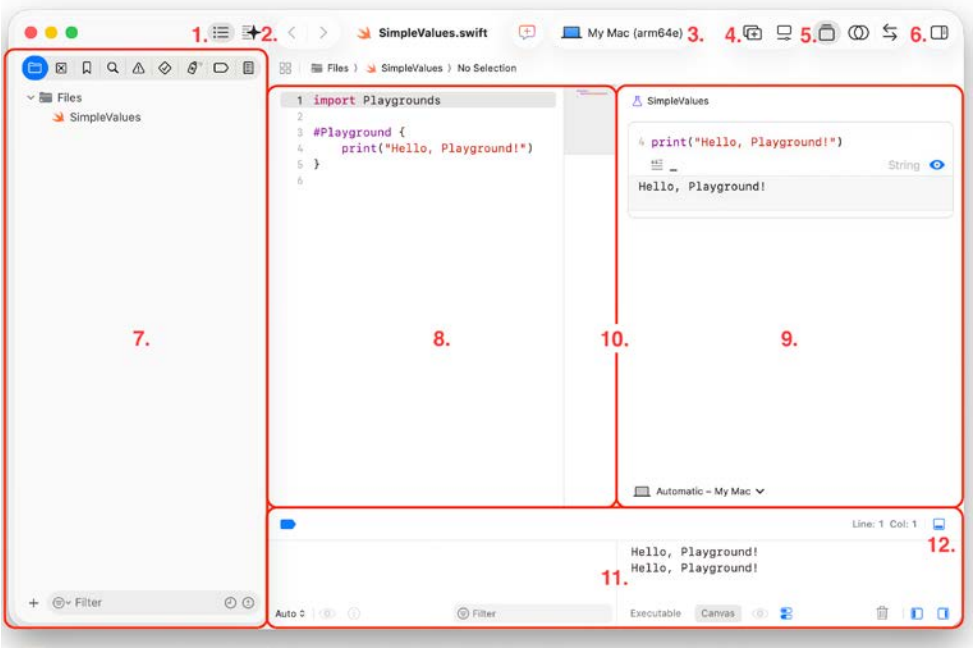


Figure 2.4: Xcode playground user interface

Let's look at the interface in more detail:

- **Navigator button (1)**: This shows or hides the Navigator area.
- **Coding Assistant button (2)**: This shows or hides the Coding Assistant area. This feature helps you write code and fix issues with Xcode's integrated intelligence.
- **Destination menu (3)**: This shows the current device or simulator used to run the code in the playground, and allows you to select a different device or simulator.
- **Editor buttons (4)**: The left button allows you to add Editor tabs or Editor panes, and the right button allows you to adjust Editor options.
- **Editor mode chooser (5)**: The left button displays previews and playgrounds in the Canvas area. The middle button reveals related content in the Assistant Editor. The right button enters a mode to review source control changes.
- **Inspector button (6)**: Used to display and hide the Inspector area.
- **Navigator area (7)**: This provides quick access to various parts of your project. The Project navigator is displayed by default.
- **Editor area (8)**: You write code here.
- **Canvas area (9)**: This provides immediate feedback on the code you write.
- **Border (10)**: This border separates the Editor and Canvas areas. You can drag the border to the left or right to change the space allocated to each area.
- **Debug area (11)**: This displays the results of the `print()` command.
- **Debug button (12)**: This shows and hides the Debug area.

You may find the text in the playground too small and hard to read. Let's see how to make it larger in the next section.

Change fonts and colors

Xcode has extensive customization options available. You can access them in the **Settings...** menu. If you find that the text in the playground is small and hard to see, follow these steps:

1. Choose **Settings...** from the Xcode menu to display the **Settings** window.
2. In the **Settings** window, click **Appearance**, click the **Fonts** tab, and increase the font size to make the text larger and easier to read:

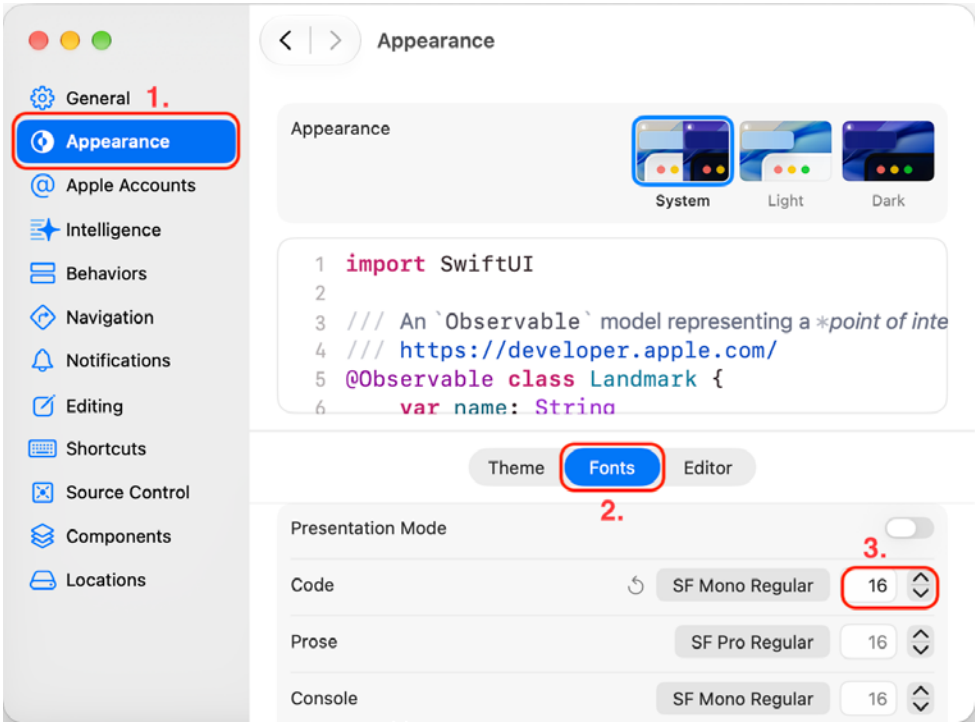


Figure 2.5: Xcode settings window with Presentation Mode enabled in the Fonts tab

3. Close the **Settings** window to return to the playground. Note that the text in the playground is larger than it was before. You can also try other themes if you'd like.

Now that you've customized the fonts and colors to your liking, let's see how to use a playground to run Swift code in the next section.

Run playground code

Your playground is a Swift file that contains a playground macro. Any Swift code between the curly braces of this macro will be automatically executed, and the results will appear in the Canvas area:

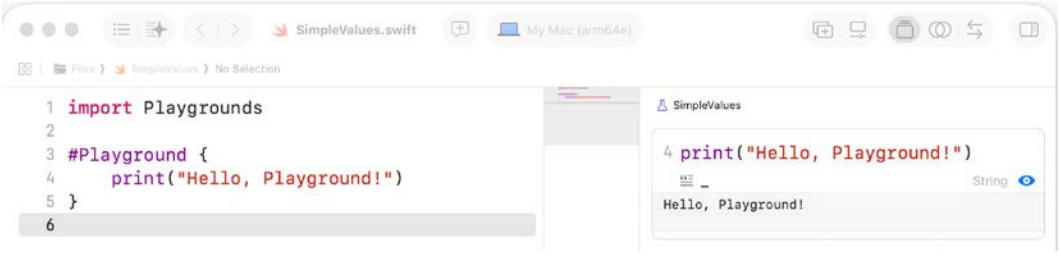


Figure 2.6: Playground showing "Hello, Playground!" in the Canvas area



For more information about the playground macro, see this link: `https://developer.apple.com/documentation/Xcode/running-code-snippets-using-the-playground-macro`.

To prepare the playground for use in the remainder of this chapter, delete the `print("Hello, Playground!")` statement from the playground. As you go along, type the code shown in this chapter between the curly braces of the playground macro, and it will run automatically, displaying the results in the Canvas area.

Let's dive into the simple data types used in Swift in the next section.

Data types

Most programming languages provide ways to work with numbers, words, and true/false values, and Swift is no different. Even if you're an experienced programmer, you may find that Swift represents these values differently from other languages that you may be familiar with.



For more information on data types, visit `https://docs.swift.org/swift-book/documentation/the-swift-programming-language/thebasics`.

Let's walk through the Swift versions of **integers**, **floating-point numbers**, **strings**, and **Booleans** in the next sections.

Integers

Let's say you want to store the following:

- The number of restaurants in a city
- Passengers in an airplane
- Rooms in a hotel

You would use integers, which are numbers without a fractional component (including negative numbers). Integers in Swift are represented by the `Int` type.

Floating-point numbers

Let's say you want to store the following:

- Pi (3.14159...)
- Absolute zero (-273.15°C)

You would use floating-point numbers, which are numbers with a fractional component (including negative numbers). Floating-point numbers in Swift are represented by the `Double` type (short for double precision floating-point number) by default.

Strings

Let's say you want to store the following:

- The name of a restaurant, such as "Bombay Palace"
- A job description, such as "Accountant" or "Programmer"
- A kind of fruit, such as "banana"

You would use Swift's `String` type, which represents a sequence of characters and is fully Unicode-compliant. This makes it easy to represent text from many writing systems and languages.



To learn more about Unicode, visit this link: `https://home.unicode.org/basic-info/faq/`.

Boolean

Let's say you want to store answers to simple yes/no questions, such as the following:

- Is it raining?
- Are there any available seats at the restaurant?

You would use Swift's `Bool` type, which represents logical Boolean values, and can only be assigned either `true` or `false`.



Boolean values are named after George Boole, a 19th century English mathematician. For more information, see this link: `https://en.wikipedia.org/wiki/Boolean_data_type`.

Now that you know how Swift represents these common data types, let's try them out in the playground you created earlier in the next section.

Use common data types in the playground

When you type code into a playground, the results will almost immediately appear in the Canvas area. Let's see what happens when you type numbers, strings, and Boolean values into your playground. Follow these steps:

1. Type the following code into the Editor area of your playground:

```
// SimpleValues
42
-23
3.14159
0.1
-273.15
“hello, world”
“albatross”
true
false
```

   Note that any line preceded by `//` is a **comment**. Comments are a great way to create notes or reminders for yourself and will be ignored by Xcode.

2. After a few seconds, Xcode will evaluate your code and display results in the Canvas area, as shown:

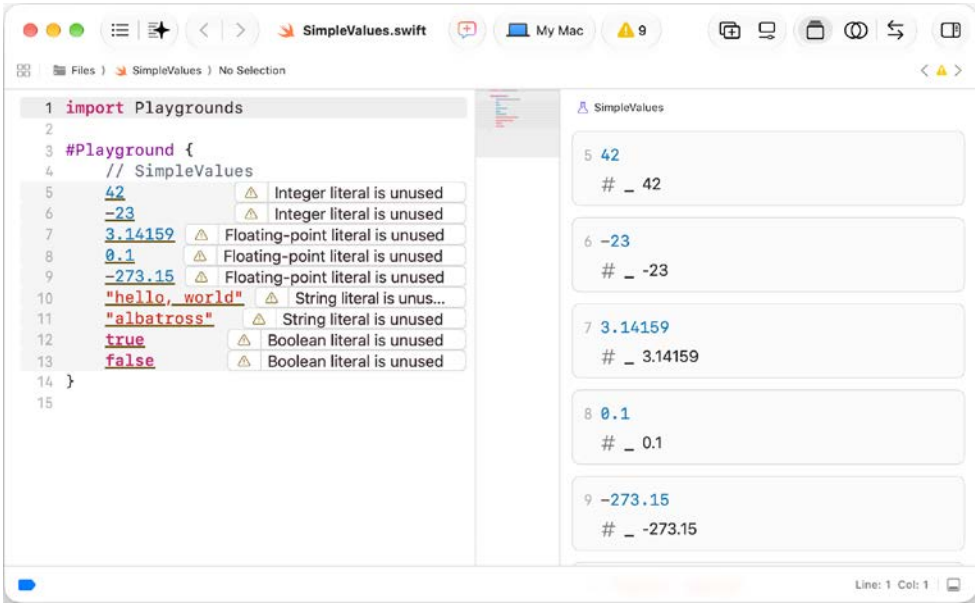


Figure 2.7: Playground showing results in the Canvas area

Note that comments do not appear in the Canvas area, and each subsequent line of code after the comment has a warning notification (a yellow triangle containing an exclamation mark). This means Xcode is telling you that something might be wrong with your code, but it will still run. In this case, it just means that the values that you typed in are not being used, so you can disregard the warning notifications for now.

Cool! You have just created your first playground. Let's look at how to store different data types in the next section.

Constants and variables

Now that you know about the simple data types that Swift supports, let's look at how to store them so that you can perform operations on them later.

You can use **constants** or **variables** to store values. Both are containers that have a name, but a constant's value can only be set once and cannot be changed after it is set, whereas a variable's value can be changed at any time.

You must declare constants and variables before you use them. Constants are declared with the `let` keyword while variables are declared with the `var` keyword.

Let's explore how constants and variables work by implementing them in your playground. Follow these steps:

1. Add the following code after all other code in your playground to declare three constants:

```
let theAnswerToTheUltimateQuestion = 42
let pi = 3.14159
let myName = "Ahmad Sahar"
```

In each case, a container is created and named, and the assigned value is stored in it. Note that a sequence of characters enclosed by double quotation marks, `"Ahmad Sahar"`, is used to assign the value for `myName`. This is known as a **string literal**.

You may have noticed that the names of constants and variables shown here start with a lowercase letter, and if there is more than one word in the name, every subsequent word starts with a capital letter. This is known as **camel case**. Doing this is strongly encouraged, as most experienced Swift programmers adhere to this convention.

1. Add the following code after the constant declarations to declare three variables:

```
var currentTemperatureInCelsius = 27
var myAge = 50
var myLocation = "home"
```

Like constants, a container is created and named in each case, and the assigned value is stored in it.

2. The value of a constant can't be changed once it is set. To test this, add the following code after the variable declarations:

```
let isRaining = true
isRaining = false
```

As you type the second line of code, a pop-up menu will appear with suggestions:

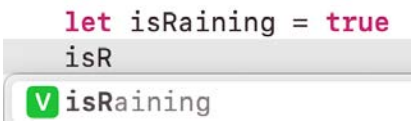


Figure 2.8: Autocomplete pop-up menu

Use the up and down arrow keys to choose the `isRaining` constant and press the *Tab* key to select it. This feature is called **autocomplete** and helps to prevent typing mistakes when you enter code.

3. When you finish typing, wait a few seconds. On the second line, you'll see an error notification (a red octagon containing an exclamation mark) appear:

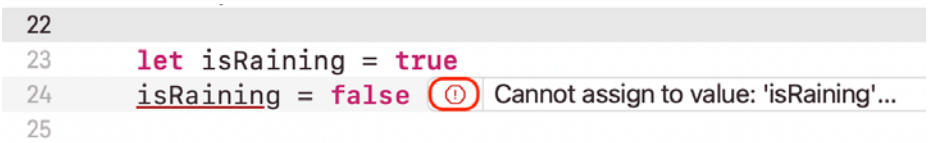


Figure 2.9: Error notification

This means there is an error in your program. The error appears because you are trying to assign a new value to a constant after its initial value has been set.

4. Click the notification to expand the error message. You'll see the following box with an **Apply** button:

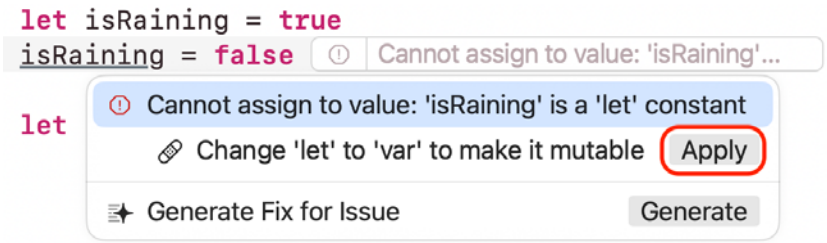


Figure 2.10: Expanded error notification

Xcode tells you what the problem is (**Cannot assign to value: 'isRaining' is a 'let' constant**) and suggests a correction (**Change 'let' to 'var' to make it mutable**). "Mutable" just means that the value can be changed after it has been set initially.

5. Click the **Apply** button. You'll see that the `isRaining` constant declaration has been changed to a variable declaration:

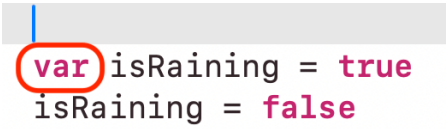


Figure 2.11: Code with a fix applied

Since a new value can be assigned to a variable after it has been created, the error is resolved. Do note, however, that the suggested correction might not be the best solution. As you gain more experience with iOS development, you'll be able to determine the best course of action.

If you look at the code you typed in, you might wonder how Xcode knows the type of data stored in a variable or constant. You'll learn how that is done in the next section.

Type inference and type safety

In the previous section, you declared constants and variables and assigned values to them. Xcode automatically determines the constant or variable type based on the value provided. This is called **type inference**. You can see the type of a constant or variable by holding down the *Option* key and clicking its name. To see this in action, follow these steps:

1. Add the following statement after all other code in your playground to declare a string:

```
let cuisine = "American"
```

2. Hold down the *Option* key and click `cuisine` to reveal the constant type. You should see the following:

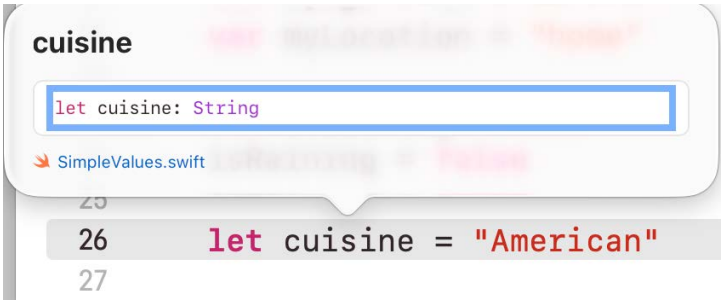


Figure 2.12: Type declaration displayed

As you can see, `cuisine` is of type `String`.

What if you want to set a specific type for a variable or constant? You'll see how to do that in the next section.

Use type annotation to specify a type

You've seen that Xcode tries to automatically determine the data type of a variable or constant based on the value provided. However, at times, you may wish to specify a type instead of letting Xcode do it for you. To do this, type a colon (`:`) after a constant or variable name, followed by the desired type. This is known as **type annotation**.

Add the following statement after all other code in your playground to declare a variable, `restaurantRating`, of type `Double`:

```
var restaurantRating: Double = 3
```

Here, you specified that `restaurantRating` has a specific type, `Double`. Even though you assigned an integer, `3`, to `restaurantRating`, it will be stored as `3.0`, a floating-point number.

In the next section, you'll learn how Xcode helps you reduce the number of errors in your program by enforcing type safety.

Use type safety to check values

Swift is a type-safe language. Xcode checks to see whether you're assigning values of the correct type to variables and flags mismatched types as errors. Let's see how this works by following these steps:

1. Add the following statement after all other code in your playground to assign a string to `restaurantRating`:

   ```
   restaurantRating = “Good”
   ```

2. You'll see an error notification (a red octagon with an exclamation mark inside it). Click on the error notification.
3. Since you are trying to assign a string to a variable of type `Double`, the following error message is displayed:

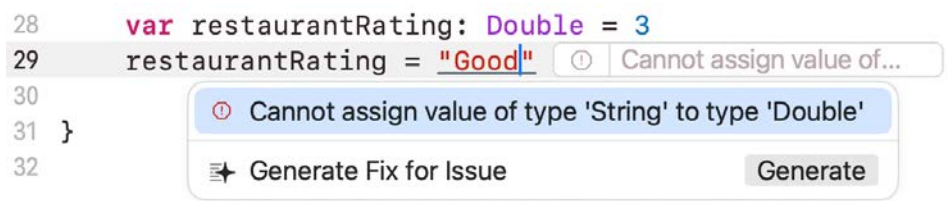


Figure 2.13: Expanded error notification

4. Comment out the line by typing `//` before it, as shown here:

   ```
   // restaurantRating = “Good”
   ```

   The error notification disappears as there are no more errors in your program.



Select lines of code and type *Command* + / to comment them out.

Now that you know how to store data in constants and variables, let's look at how to perform operations on them in the next section.

Operators

You can perform arithmetic, comparison, and logical operations in Swift. **Arithmetic operators** are used to perform common mathematical operations. **Comparison** and **logical operators** check an expression's value and return `true` or `false`. The values that operators affect are **operands**.

For more information on operators, visit `https://docs.swift.org/swift-book/documentation/the-swift-programming-language/basicoperators`.

Let's look at each operator type in more detail. You'll start with arithmetic operators (addition, subtraction, multiplication, and division) in the next section.

Arithmetic operators

You can perform mathematical operations on integer and floating-point numbers by using the standard arithmetic operators shown here:

+	Addition
-	Subtraction
*	Multiplication
/	Division

Figure 2.14: Arithmetic operators

Let's see how these operators are used. Follow these steps:

1. Add the following arithmetic operations after all other code in your playground:

```
let sum = 23 + 20
let result = 32 - sum
let total = result * 5
let divide = total / 10
```

2. Verify that the results displayed in the Results area will be `43`, `-11`, `-55`, and `-5`, respectively. Note that `-55` divided by `10` returns `-5` instead of `-5.5`, as both operands are integers, and integer division truncates the fractional part.
3. Enter the following code to see what happens if the operands in an arithmetic expression are of different types:

```
let a = 12
let b = 12.0
let c = a + b
```

   You'll get an error message (**Binary operator '+' cannot be applied to operands of type 'Int' and 'Double'**). This is because `a` and `b` are different types. Note that Xcode can't fix this automatically, so it does not display any fix-it suggestions.

4. To fix the error, modify the last statement as follows:

```
let c = Double(a) + b
```

   `Double(a)` converts the `Int` value stored in `a` into a `Double` value. Both operands are now of the same type, and now you can add it to the value in `b`. The value assigned to `c` is `24.0`, and `24` will be displayed in the Results area.

Now that you know how to use arithmetic operators, you'll look at compound assignment operators (+=, -=, *=, and /=) in the next section.

Compound assignment operators

You can perform an operation on a value and assign the result to a variable using the compound assignment operators shown here:

+=	Adds a value and assigns the result to the variable
-=	Subtracts a value and assigns the result to the variable
*=	Multiplies by the value and assigns the result to the variable
/=	Divides by the value and assigns the result to the variable

Figure 2.15: Compound assignment operators

Let's see how these operators are used. Add the following code after all other code in your playground:

```
var d = 1
d += 2
d -= 1
```

The `d += 2` expression is shorthand for `d = d + 2`, so the value in `d` is now `1 + 2`, and `3` will be assigned to `d`. In the same way, `d -= 1` is shorthand for `d = d - 1`, so the value in `d` is now `3 - 1`, and `2` will be assigned to `d`.

Now that you are familiar with compound assignment operators, let's look at comparison operators (==, !=, >, <, >=, and <=) in the next section.

Comparison operators

You can compare one value to another using comparison operators, and the result will be true or false. You can use the following comparison operators:

==	Equal to
!=	Not equal to
>	Greater than
<	Less than
>=	Greater than or equal to
<=	Less than or equal to

Figure 2.16: Comparison operators

Let's see how these operators are used. Add the following code after all other code in your playground:

```
1 == 1
2 != 1
2 > 1
1 < 2
1 >= 1
2 <= 1
```

Let's see how this works:

- `1 == 1` returns `true` because 1 is equal to 1.
- `2 != 1` returns `true` because 2 is not equal to 1.
- `2 > 1` returns `true` because 2 is greater than 1.
- `1 < 2` returns `true` because 1 is less than 2.
- `1 >= 1` returns `true` because 1 is greater than or equal to 1.
- `2 <= 1` returns `false` because 2 is not less than or equal to 1.

The returned Boolean values will be displayed in the Results area.

What happens if you want to check more than one condition? That's where logical operators (**AND**, **OR**, and **NOT**) come in. You'll learn about those in the next section.

Logical operators

Logical operators are handy when you deal with two or more conditions. For example, if you are at a convenience store, you can pay for items if you have cash or a credit card. OR is the logical operator in this case. You can use the following logical operators:

&&	Logical AND - returns true only if all conditions are true
||	Logical OR - returns true if any condition is true
!	Logical NOT - returns the opposite Boolean value

Figure 2.17: Logical operators

To see how these operators are used, add the following code after all other code in your playground:

```
(1 == 1) && (2 == 2)
(1 == 1) && (2 != 2)
(1 == 1) || (2 == 2)
(1 == 1) || (2 != 2)
(1 != 1) || (2 != 2)
!(1 == 1)
```

Let's see how this works:

- `(1 == 1) && (2 == 2)` returns `true` as both operands are `true`, so `true AND true` returns `true`.
- `(1 == 1) && (2 != 2)` returns `false` as one operand is `false`, so `true AND false` returns `false`.
- `(1 == 1) || (2 == 2)` returns `true` as both operands are `true`, so `true OR true` returns `true`.
- `(1 == 1) || (2 != 2)` returns `true` as one operand is `true`, so `true OR false` returns `true`.
- `(1 != 1) || (2 != 2)` returns `false` as both operands are `false`, so `false OR false` returns `false`.
- `!(1 == 1)` returns `false` as `1==1` is `true`, so `NOT true` returns `false`.

The returned Boolean values will be displayed in the Canvas area.

So far, you've only worked with numbers. In the next section, you'll see how you can perform operations on words and sentences, which are stored as strings using Swift's `String` type.

String operations

As you saw earlier, a string is a series of characters. They are represented by the `String` type, and they are fully Unicode-compliant.

For more information on strings, visit `https://docs.swift.org/swift-book/documentation/the-swift-programming-language/stringsandcharacters`.

Let's learn about some common string operations. Follow these steps:

1. You can join two strings together using the + operator. Add the following statement after all other code in your playground:

   ```
   let greeting = “Good” + “ Morning”
   ```

   The values of the string literals `"Good"` and `" Morning"` are joined together, and `"Good Morning"` is displayed in the Canvas area.

2. You can combine strings with constants and variables of other types by making them strings as well. To change a constant, `rating`, into a string, enter the following statements:

   ```
   let rating = 3.5
   var ratingResult = “The restaurant rating is “ + String(rating)
   ```

The `rating` constant contains `3.5`, a value of type `Double`. Putting `rating` between the brackets of `String()` gets the value stored in `rating` and creates a new string based on it, `"3.5"`, which is combined with the string in the `ratingResult` variable, returning the string `"The restaurant rating is 3.5"`.

3. There is a simpler way of combining strings called **string interpolation**. String interpolation is done by typing the name of a constant or variable between `"\("` and `")"` in a string. Enter the following statement:

```
ratingResult = “The restaurant rating is \(rating)”
```

As in the previous example, the value in `rating` is used to create a new string, `"3.5"`, returning the string `"The restaurant rating is 3.5"`.

So far, you can see the results of your instructions in the Canvas area. However, when you write your app using Xcode, you won't have access to the Canvas area that you see in your playground. To display the contents of variables and constants while your program runs, you'll learn how to print them to the Debug area in the next section.

The print() statement

As you saw in *Chapter 1, Hello, Xcode*, an Xcode project and an Xcode playground both have a Debug area. Using the `print()` statement will print anything between the brackets to the Debug area.

The `print()` statement is a function. You'll learn more about functions in *Chapter 6, Functions and Closures*.

Add the following statement after all other code in your playground:

```
print(ratingResult)
```

You'll see the value of `ratingResult` appear in the Debug area:



Figure 2.18: The Debug area showing the result of the print() statement

When you're just starting out, feel free to use as many `print()` statements as you like. It's a really good way to understand what is happening in your program.

Summary

In this chapter, you learned how to create and use playground files, which allow you to explore and experiment with Swift.

You saw how Swift represents different types of data, and how to use constants and variables. This enables you to store numbers, Boolean values, and strings in your program.

You also learned about type inference, type annotation, and type safety, which help you to write code concisely and with fewer errors.

You looked at how to perform operations on numbers and strings, which lets you perform simple data-processing tasks.

You learned how to fix errors and how to print to the Debug area, which is useful when you're trying to find and fix errors in the programs that you write.

In the next chapter, you'll look at **conditionals** and **optionals**. Conditionals deal with making logical choices in your program, and optionals deal with cases where a variable may or may not have a value.